

PhantomGrid

AI-Driven Passive Behavioural Authentication Engine

CBI Hackathon 2026 - Phase II Technical Documentation

Team ZeroIntent | S.No. 8 | IIIT Kottayam

Madapati Jyoti Radithya | Kontheti Sai Akhilesh | Praising Y Harris Ratnam (Lead)

1. Problem Statement

Account Takeover (ATO) fraud in Public Sector Banks (PSBs) has reached a critical inflection point. In FY2023, Indian PSBs reported over INR 7,400 crore in digital fraud losses - the majority occurring after successful login. Attackers bypass authentication entirely using stolen credentials purchased from dark-web marketplaces.

The fundamental failure of current authentication is temporal: systems verify identity once at the login gate, then trust the session unconditionally. An attacker with valid credentials - obtained through phishing, SIM swap, or social engineering - is indistinguishable from a legitimate user by any existing control.

PhantomGrid answers the question that matters at the point of a fund transfer: Is the person currently operating this session the enrolled account holder? It answers continuously, silently, and with zero additional friction.

2. Proposed Solution

PhantomGrid is a three-layer passive behavioural authentication engine running transparently beneath a banking portal. Rather than challenging users with additional factors, it observes unconscious behavioural signatures that are stable over time and practically impossible to replicate.

The system operates in two phases:

Enrollment (Sessions 1-5): PhantomGrid silently collects behavioural baseline vectors across three independent signal domains during normal transactions. No user action required.

Continuous Scoring (Session 6+): Every transaction is scored against the enrolled baseline. A composite risk score (0-100) determines the response: ALLOW, OTP step-up, or BLOCK.

The analyst dashboard provides real-time visibility into session scores, per-layer risk breakdown, explainability reasoning, and a tamper-evident audit trail.

3. System Architecture

PhantomGrid follows a three-tier architecture with clear separation of concerns:

TIER 1: Signal Capture (Madapati Jyoti Radithya)

capture.js - behavioural hooks injected into the banking portal
Signals: decoy_tap_count, bene_dwell_ms, amount_iki[], pin_vector[]
Packages all signals into ONE JSON payload per transaction

TIER 2: ML Inference Engine - FastAPI + Python 3.12 (Kontheti Sai Akhilesh)

Layer 1 CognitiveTrap: Isolation Forest on decoy interactions
Layer 2 IntentTrace: Isolation Forest on navigation behaviour
Layer 3 RhythmLock: Dynamic Time Warping on PIN keystroke rhythm
Fusion: $\text{composite} = L1 \times 0.30 + L2 \times 0.40 + L3 \times 0.30$

Security: SHA-256 replay defence + hash-chain audit log
Decision: ALLOW (<60) | OTP (60-79) | BLOCK (>=80)

TIER 3: Analyst Dashboard (Praising Y Harris Ratnam)

Polls GET /logs every 2 seconds

Live gauge, layer bars, trend chart, fraud alert, audit badge

4. Technology Stack

Component	Technology	Rationale
API Framework	FastAPI (Python 3.12)	Async, auto-validates with Pydantic, Swagger UI
Anomaly Detection (L1,L2)	scikit-learn IsolationForest	Unsupervised - no fraud labels needed at enrollment
Rhythm Comparison (L3)	dtaidistance (DTW)	Handles natural speed variation in keystroke sequences
Database ORM	SQLAlchemy 2.0	Declarative models, PostgreSQL-compatible schema
Database	SQLite / PostgreSQL	SQLite for PoC; Railway uses env-var for PostgreSQL
Validation	Pydantic v2	Zero-overhead schema enforcement, 422 on malformed input
Audit Crypto	hashlib SHA-256	Hash chain + payload signatures for replay defence
Frontend	Vanilla HTML/CSS/JS	No build step - fully portable, zero dependency
Tests	pytest + requests	8 integration tests against live backend
Deployment	Railway (Nixpacks)	Live at phantomgrid-production.up.railway.app

5. Workflow

5.1 Enrollment Flow

```
User initiates payment
capture.js collects: decoy_tap_count, amount_hesitations (L1)
                    bene_dwell_ms, amount_iki[] (L2)
                    pin_vector[] - inter-key intervals ONLY, no digits (L3)
POST /enroll {user_id, all signals}
Backend stores feature vectors in user_profiles (JSON blobs)
After 5 samples: enrollment complete, capture.js auto-switches to VERIFY mode
```

5.2 Verification / Scoring Flow

```
POST /verify {user_id, signals}
1. Replay check: SHA-256(payload) in seen_signatures (5-min window)?
   YES -> forced BLOCK, replay_detected: true
   NO  -> continue
2. Layer scoring:
   L1: continuous_if_risk(baseline, [decoy_taps, hesitations])
   L2: continuous_if_risk(baseline, [bene_dwell, avg_iki])
   L3: dtw_to_risk(min DTW distance to any enrolled PIN vector)
3. Fusion: composite = L1*0.30 + L2*0.40 + L3*0.30
4. Decision: ALLOW (<60) | OTP (60-79) | BLOCK (>=80)
5. If ALLOW: append to baseline (adaptive learning, window 20)
6. Hash chain: row_hash = SHA256(prev_hash|session_data)
7. Write session_logs, return scores + decision + replay_detected
8. Dashboard polls GET /logs every 2s -> updates live
```

6. Database Design

Table: user_profiles

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
user_id	TEXT UNIQUE	User identifier (from bank session)
layer1_vectors	TEXT (JSON)	List of [decoy_tap_count, amount_hesitations] pairs
layer2_vectors	TEXT (JSON)	List of [bene_dwell_ms, avg_amount_iki] pairs
pin_vectors	TEXT (JSON)	List of PIN inter-key interval vectors (no digits)

Table: session_logs

Column	Type	Description
id	INTEGER PK	Auto-increment
session_id	TEXT UNIQUE	UUID-derived 8-char session identifier
timestamp	DATETIME	UTC timestamp of session
user_id	TEXT	User who initiated session
layer1_score	REAL	CognitiveTrap risk score (0-100)
layer2_score	REAL	IntentTrace risk score (0-100)
layer3_score	REAL	RhythmLock risk score (0-100)
composite_score	REAL	Fused weighted risk score (0-100)
decision	TEXT	ALLOW OTP BLOCK
row_hash	TEXT	SHA-256 of this row - tamper detection
prev_hash	TEXT	SHA-256 of preceding row - hash chain link

7. AI / ML Models

7.1 Isolation Forest - Layers 1 & 2 (CognitiveTrap, IntentTrace)

Isolation Forest (Liu et al., 2008) is an unsupervised anomaly detection algorithm that exploits the property that anomalies are rare and different. It builds 100 isolation trees by recursively partitioning the feature space with random splits. Normal points require many splits to isolate (deep paths). Anomalies require few (shallow paths = anomaly score).

PhantomGrid extends standard IF with a continuous hybrid engine (services/scoring.py) that solves the saturation problem on small (5-sample) per-user baselines:

```
def continuous_if_risk(training_data, point):
    deviation = _normalized_deviation(point, training_data)
    if len(training_data) < 10:          # Small baseline: skip IF gate
        return round(min(100.0, deviation * 30.0), 1)
    model = IsolationForest(contamination=0.1, random_state=42).fit(training_data)
    gate = model.decision_function([point])[0]  # >0 inlier, <0 outlier
    if gate >= 0:
        return round(min(45.0, deviation * 22.0), 1)  # Inlier: 0-45
    return round(min(100.0, 55.0 + deviation * 12.0), 1)  # Outlier: 55-100
```

Hyperparameters: contamination=0.1 (10% noise tolerance), n_estimators=100, random_state=42.

Small-baseline fallback: For baselines under 10 samples, pure normalized-deviation scoring is used - monotonic, deterministic, and immune to IF instability on sparse data.

7.2 Dynamic Time Warping - Layer 3 (RhythmLock)

DTW (Sakoe & Chiba, 1978) finds the optimal elastic alignment between two time series before measuring residual distance. This is critical for keystroke dynamics: a user typing their PIN 10% faster when stressed still produces the same relative rhythm, which Euclidean distance would incorrectly flag as anomalous.

Implementation: DTW distance computed between query PIN vector and every enrolled baseline vector. Minimum distance taken (best-match strategy, reduces FRR). Mapped to risk score:

```
def dtw_to_risk(distance):
    return round(min(100.0, (distance / 180.0) * 100.0), 1)
```

Calibration: 180ms total DTW deviation saturates to full risk (100). Legit jitter: 0-10ms -> score 0-6.

7.3 Adaptive Learning

After every ALLOW decision, the session's behavioural vectors are appended to the user's baseline (sliding window, capped at 20 samples per layer). This implements online incremental learning - the model drifts toward the user's current behaviour, handling natural evolution over time (new device, aging, lifestyle changes) without explicit retraining.

7.4 Performance Benchmark

Validated on 200 synthetic sessions (100 legitimate, 100 attacker):

Metric	Result	Interpretation
Detection Rate (TPR)	96.7%	Of 100 attacker sessions, 97 correctly blocked
False Positive Rate (FPR)	0.0%	Zero legitimate users incorrectly blocked
AUC (ROC Curve)	1.00	Perfect separation of legit vs attacker populations

8. APIs and External Services

PhantomGrid uses no external APIs or cloud ML services. All computation is self-contained.

Endpoint	Method	Description
----------	--------	-------------

/enroll

POST

Store one enrollment sample (5 needed to complete baseline)

/verify

POST

Score session: returns {layer scores, composite, decision, replay_detected}

/logs?user_id=X

GET

Last 20 session log rows, optionally filtered by user

/maturity?user_id=X

GET

Baseline maturity: {samples, required, mature, confidence}
--

/audit/verify

GET

Walk hash chain: {valid, verified, broken_at_session}

9. Security Measures

9.1 Replay-Attack Defence

SHA-256 signature computed over the full behavioural payload (user_id, all signal values). Exact duplicate signatures within a 5-minute window are detected by services/audit.py::is_replay() and forced to BLOCK with replay_detected: true in the response. This prevents attackers from capturing a legitimate session packet and retransmitting it verbatim.

9.2 Tamper-Evident Audit Chain

SHA-256 hash chain over session_logs. Each row contains row_hash (SHA-256 of its own content including the previous row's hash) and prev_hash (the prior row's hash). Any retrospective modification of any single record invalidates the chain from that point forward, detectable instantly via GET /audit/verify. Satisfies RBI Master Circular requirements for immutable transaction audit trails.

9.3 Input Validation & PII Minimisation

All API requests are validated via Pydantic v2 schemas before any ML code executes. Malformed payloads receive HTTP 422. No SQL injection surface (SQLAlchemy ORM with parameterised queries).

PIN digits are never persisted. Only inter-keystroke timing intervals (millisecond gaps between digit entries) are stored. These values cannot be reverse-engineered to recover the PIN. No account numbers, names, or transaction amounts are stored by PhantomGrid.

9.4 STRIDE Threat Coverage

Threat	Control	Status
Spoofing	Per-user IF models - impersonation requires exact behavioral mimicry	Mitigated
Tampering	SHA-256 hash chain - any DB edit is detectable at exact row	Mitigated
Repudiation	Immutable session_logs with UTC timestamps and hash chain	Mitigated
Info Disclosure	No PII stored; behavioral vectors non-reversible	Mitigated
Denial of Service	Lightweight inference <5ms; rate-limiting in production	Partial
Elevation of Privilege	/enroll requires authenticated session in production	PoC gap

10. Scalability Considerations

Concern	PoC Approach	Production Path
Database	SQLite (single file)	PostgreSQL - drop-in connection string swap
Model storage	Fit on-the-fly per request	Pre-serialised IF models (joblib) + Redis cache
Dashboard updates	REST polling every 2s	WebSocket streaming at 50ms
Deployment	Railway (single region)	Docker + AWS ap-south-1 (RBI data localisation)
Baseline training	5 enrollment samples	Shadow mode 30+ samples before enforcement
Inference latency	L1+L2+L3 < 5ms total	Unchanged - does not block payment flow

11. Assumptions and Limitations

Assumptions

1. Users enroll on the same device used for live transactions. PIN rhythm is device-specific (keyboard type affects timings).
2. At least 5 enrollment samples are available before scoring enforcement begins.
3. The /enroll endpoint is called within an authenticated banking session (prevents enrollment poisoning - flagged as production requirement in threat model).
4. capture.js has not been tampered with by a client-side attacker (XSS vector).

Limitations

1. Cold Start: Under 5 enrollment samples, model reliability is reduced. The /maturity endpoint signals this explicitly to the analyst dashboard.
2. Cross-Device Degradation: PIN rhythm changes significantly across physical keyboards. Mitigation: per-device baseline profiles (production roadmap).
3. Sustained Adaptive Poisoning: A sophisticated attacker achieving repeated OTP decisions over many sessions could gradually drift the baseline. Mitigation: drift detection with exponential moving average (production roadmap).
4. SQLite Concurrency: Single-writer limitation. Not suitable for high-concurrency production without migration to PostgreSQL.

12. Future Enhancements

Feature	Description	Priority
Mouse/Touch Dynamics	L4 layer: scroll velocity, touch pressure, pointer trajectory entropy	High
Device-Aware Profiles	Separate baseline per device fingerprint - eliminates cross-device FRR spike	High
PostgreSQL Migration	Drop-in connection string change; enables horizontal scaling	High
Mobile SDK	Touch pressure, swipe velocity, accelerometer during PIN entry	High
WebSocket Streaming	Real-time dashboard at 50ms vs 2s polling	Medium
Federated Baseline	Global population model on anonymised data to bootstrap cold-start users	Medium
Drift Detection	Statistical test on baseline evolution - alert on unnatural drift	Medium
RBI Reporting Module	Per-user risk history, aggregate metrics, audit export for regulators	Medium
Active Deception Layer	Randomise decoy positions per session - hardens against layout memorisation	Low

13. Live Deployment

PhantomGrid is deployed live on Railway (cloud platform) at:

API Base: <https://phantomgrid-production.up.railway.app>

Swagger UI: <https://phantomgrid-production.up.railway.app/docs>

GitHub: <https://github.com/Pyhroff/PhantomGrid> (private, shared with CBIHack26)

The deployment auto-seeds demo_user on startup. Judges can immediately POST to /verify with user_id: demo_user without any setup. The backend serves all endpoints documented above.

Team

Role	Name	Contribution
Frontend - Signal Capture	Madapati Jyoti Radithya	NexaBank portal + capture.js behavioural hooks
Backend - ML Engine	Kontheti Sai Akhilesh	FastAPI + IsolationForest + DTW + SQLite
Dashboard + Security + Tests (Lead)	Praising Y Harris Ratnam	Dashboard, audit chain, replay defence, test suite

PhantomGrid - Passive. Invisible. Unbeatable.

Team ZeroIntent | S.No. 8 | CBI Hackathon 2026 | MNNIT Allahabad | IIIT Kottayam